

Deep Learning: Building Blocks of CNNs

1. Why Not Just Use an MLP for Images?

Before understanding CNNs, we need to understand why regular MLPs (Multi-Layer Perceptrons) are a terrible choice for processing images. An MLP takes a flat vector as input, so to feed an image into it, you have to flatten it — turn the 2D grid of pixels into one long list.

Consider a modest 200×200 RGB image:

- Flattening gives: $200 \times 200 \times 3 = 120,000$ neurons just in the input layer
- One hidden layer of the same size would need: $120,000 \times 120,000 \approx 1.44 \times 10^{10}$ weights
- At 4 bytes per float, that is ≈ 58 GB of memory just for one layer — completely impractical

There are three fundamental problems with using MLPs on images:

Problem 1: Full connectivity explodes the number of weights, hitting hardware limits and causing severe overfitting.

Problem 2: Flattening **destroys the spatial structure** — nearby pixels that share visual context are treated as unrelated.

Problem 3: More data and a much more complex network would be needed to compensate.

We need a smarter approach that respects the structure of images and uses far fewer parameters.

2. The Key Insight: Translation Invariance

Natural signals like images have a very important property: a feature (say, a bird, a corner, or an edge) can appear anywhere in the image. If you train a detector for 'bird' that only works when the bird is in the top-left corner, it is useless when the bird appears elsewhere.

Remember what happens when we flatten an image for an MLP:

$200 \times 200 \times 3 \rightarrow 120,000$ numbers

Now imagine a bird appears:

Top-left → pixels 1–300
Center → pixels 60,000–60,300
Bottom → pixels 110,000–110,300

To an **MLP**, these are **completely different input patterns**.

So the network would need to learn:

bird_top_left_detector
bird_center_detector
bird_bottom_detector
bird_everywhere_detector

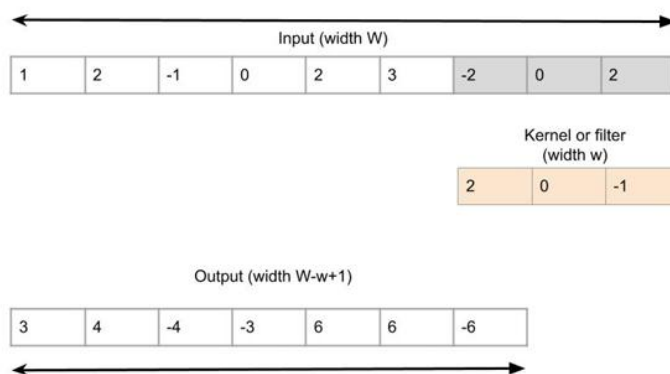
This is extremely inefficient.

This property is called translation invariance — the same pattern can occur at different locations in the signal. An ideal feature detector should respond the same way regardless of where in the image the pattern appears.

CNNs (Convolutional Neural Networks) are a class of artificial neural networks specifically designed to be shift/space invariant. They achieve this using convolution — a mathematical operation that applies the same set of weights at every location in the input, and crucially, preserves the spatial structure of the data (2D input → 2D output).

3. Convolution

1D Convolution — The Basic Idea



- Input Array is Feature Vector
- Kernel filter
- Output – Feature map
- Preserves the structure if the i/p is a 2D tensor → o/p is also a 2D tensor
- There exist a relation between the locations of i/p and o/p values

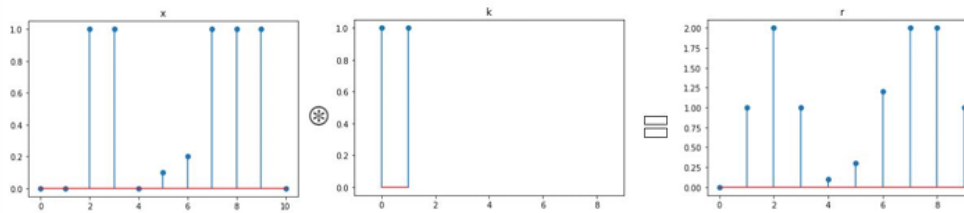
- Let $\mathbf{x} = (x_1, x_2, \dots, x_W)$ is the input, $\mathbf{k} = (k_1, k_2, \dots, k_w)$ is the kernel
- The result $(x \otimes k)$ of convolving $\mathbf{x}$ with $\mathbf{k}$ will be a 1D tensor of size $W - w + 1$

$$\begin{aligned}(x \otimes k)_i &= \sum_{j=1}^w x_{i-1+j} k_j \\ &= (x_i, \dots, x_{i+w-1}) \cdot \mathbf{k}\end{aligned}$$

Concrete Example — Edge Detection

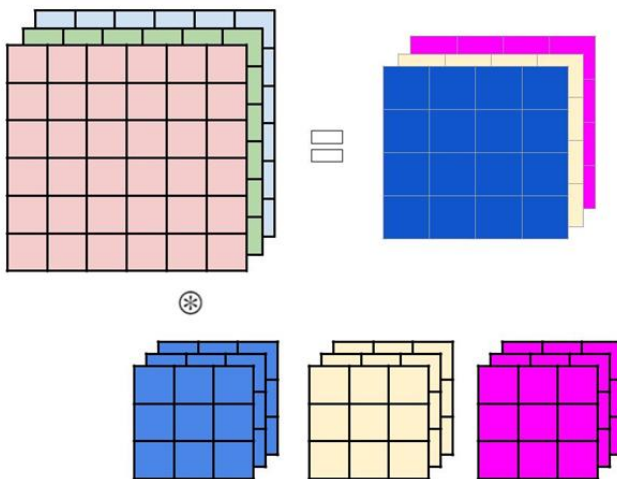
Convolution is a powerful feature extractor. A simple kernel can perform a differential (gradient) operation that detects changes in the signal. For example:

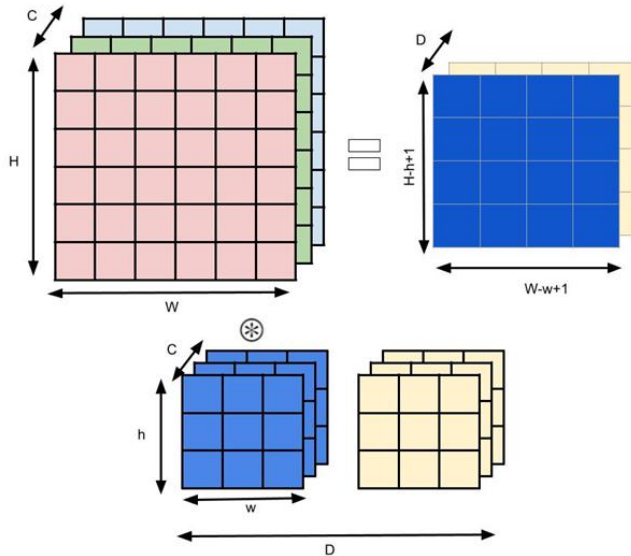
$$(0, 0, 1, 1, 0, 0.1, 0.2, 1, 1, 1, 0) \otimes (1, 1) = (0, 1, 2, 1, 0.1, 0.3, 1.2, 2, 2, 1)$$



2D Convolution — For Images

Convolution naturally extends to 2D. An image is a 2D (or 3D if you count channels) signal. The key point is that in CNNs:





- Input tensor has size: $C \times H \times W$ (Channels $\times$ Height $\times$ Width)
- Kernel tensor has size: $C \times h \times w$ (same number of channels as input, spatial size $h \times w$)
- Output tensor has size: $(H - h + 1) \times (W - w + 1)$ — a single 2D map (one channel)

The kernel is NOT convolved in the channel dimension — it slides over the spatial ($H \times W$) dimensions only. All C channels are combined into a single output value at each location. This is why we call the input a '2D signal' even though it has multiple channels: the spatial structure is 2D.

An alternative way to interpret 2D convolution: the kernel defines a small affine function that operates on an input block of size $C \times h \times w$. This same affine function is applied at every spatial location in the input — that's the key idea of weight sharing. The kernel's learned weights are the parameters of this affine function.

Structure Preservation

One of convolution's most valuable properties is that it preserves the structure of the input:

- 1D input $\rightarrow$ 1D output
- 2D input $\rightarrow$ 2D output
- Adjacent components in the output are influenced by adjacent parts in the input — spatial relationships are maintained
- If the channel dimension has a metric meaning (e.g. time in video), 3D convolution can be used across frames

4. Convolution Parameters in Detail

4a. Output Size Formula

Given an input of width W , a kernel of size k , padding p , and stride s , the output width is:

$$\text{Output} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1$$

💡 The same formula applies to height. For default (no padding, stride=1): output = $W - k + 1$.

4b. Padding

Without padding, every convolution reduces the spatial size of the output (from W to $W - k + 1$). After many layers, the feature map shrinks to almost nothing. Padding solves this by adding extra values around the border of the input before convolving.

- The most common choice is zero-padding: add rows and columns of zeros around the input
- Instead of zeros, one can also pad with the actual signal values at the edges (edge padding or reflect padding)
- 'Same' padding: choose $p = (k-1)/2$ so that the output has the same spatial dimensions as the input (when stride=1)

💡 Padding preserves spatial dimensions across layers, which is often desirable.

4c. Stride

Stride controls how far the kernel moves at each step. Default stride is 1 — the kernel moves one pixel at a time, densely covering the input. A stride of 2 means the kernel jumps 2 pixels between positions, effectively halving the output size.

- stride = 1: output size $\approx$ input size (minus border shrinkage)
- stride = 2: output size $\approx$ half of input size — commonly used instead of pooling for downsampling

4d. Dilation (Atrous Convolution)

Dilation is a way **to increase the effective size of the kernel** (its receptive field) without adding any new parameters or increasing computation proportionally. It works by inserting zeros between the kernel values, spreading them out.

- dilation = 1 (default): standard convolution, no zeros inserted
- dilation = 2: one zero inserted between each kernel element in each direction
- dilation = d: (d-1) zeros inserted between each pair of kernel elements

Effect: A kernel of size k with dilation d covers an area of size $k + (k-1)(d-1) = d(k-1) + 1$ in the input, but still only has $k \times k$ actual learned weights. This dramatically increases the receptive field — how much of the original input influences each output value — at almost no extra cost.

Dilated convolution is also called atrous convolution ('atrous' means 'with holes' in French). It is heavily used in semantic segmentation and audio models (WaveNet) where you need large receptive fields without losing spatial resolution.

5. PyTorch: Convolution in Code

- `F.conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)`
- `weight` is $D \times C \times h \times w$ dimensional kernels
- `bias` D dimensional
- `input` is $N \times C \times H \times W$ dimensional signal
- Output is $N \times D \times (H - h + 1) \times (W - w + 1)$ tensor
- Autograd compliant

```
input = torch.empty(128, 3, 20, 20).normal_()
weight = torch.empty(5, 3, 5, 5).normal_()
bias = torch.empty(5).normal_()
output = F.conv2d(input, weight, bias)
output.size()
torch.Size([128, 5, 16, 16])
```

```
weight[0,0]
tensor([[[-0.6974, 0.1342, -0.2632, -0.4672, 0.1827],
[-0.1184, -0.2164, 0.2772, -0.1099, 0.0103],
[-0.8272, 0.3580, 0.2398, -0.5795, -0.9472],
[-1.1734, -0.1019, 0.7394, 0.3342, 0.1699],
[ 1.9271, 0.1250, 0.4222, 0.2014, 1.1100]])
```

- Class `torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`
- `kernel_size` can be either a pair (h, w) or a single value k interpreted as (k, k).
- Encloses the convolution as a module
- Initializes the kernel parameters and biases as random

```
f = nn.Conv2d(in_channels = 3, out_channels = 5,
kernel_size = (2, 3))
for n, p in f.named_parameters():
...print(n, p.size())
```

```
>>weight torch.Size([5, 3, 2, 3])
>>bias torch.Size([5])
```

```
input = torch.empty(128, 3, 28, 28).normal_()
output = f(input)
output.size()
>>torch.Size([128, 5, 27, 26])
```

6. Pooling

What is Pooling?

Pooling is a down sampling operation placed between convolutional layers. It groups a small region of adjacent activations together and replaces them with a single representative value. The effect is a progressive reduction in the spatial size of the feature maps as you go deeper in the network.

- Parameter-free: pooling has no learned weights, so it adds no parameters to the model
- Uses non-overlapping stride by default (stride = kernel size)
- Also called a sub-sampling layer
- Typically placed between convolutional layers

Max Pooling

The standard pooling operation in CNNs. Takes the maximum value within each pooling window. This keeps the strongest activation in each region — if a feature detector fires strongly anywhere in the window, the output reflects that.

MaxPool output = max(values in non-overlapping window)

💡 Max pooling provides weak invariance to small local translations — moving a feature by a pixel or two within the pooling window does not change the output.

Average Pooling

Takes the average of all values in the pooling window instead of the maximum. Smoother than max pooling. Used more commonly in the final stages of a CNN (global average pooling) to reduce a feature map to a single value per channel.

AvgPool output = mean(values in non-overlapping window)

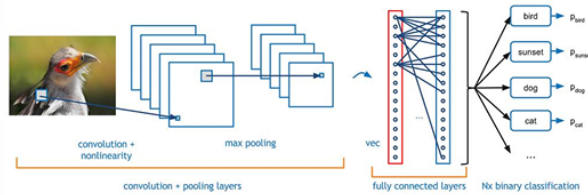
2D Pooling

In 2D (images), pooling uses a 2D window, e.g. 2×2 or 3×3.

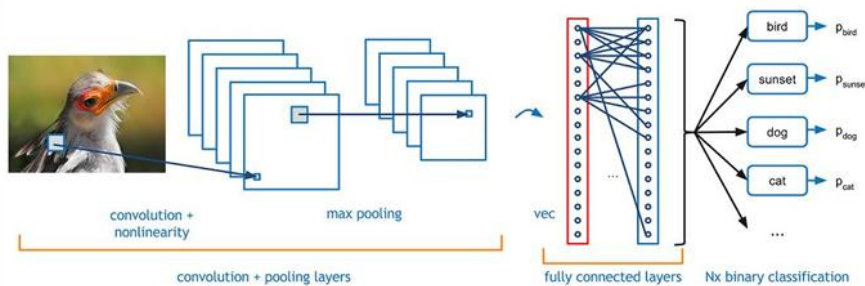
- Unlike convolution, pooling applies channel-wise — each channel is pooled independently
- This means pooling does NOT reduce the number of channels, only the spatial size (H and W)
- A 2×2 max pool halves both height and width, reducing the feature map to 1/4 of its size

7. CNN Architecture

Architecture of a simple CNN



- Initially Conv layer with nonlinearity
- Followed by a few Conv + Nonlinearity layers
- Have Pooling layers in between Conv layers → reduce the feature map size sufficiently
- Vectorize and fully connected layers



INPUT $\rightarrow$ $[[\text{CONV} \rightarrow \text{RELU}] * N \rightarrow \text{POOL}] * M \rightarrow [\text{FC} \rightarrow \text{RELU}] * K \rightarrow$
 FC